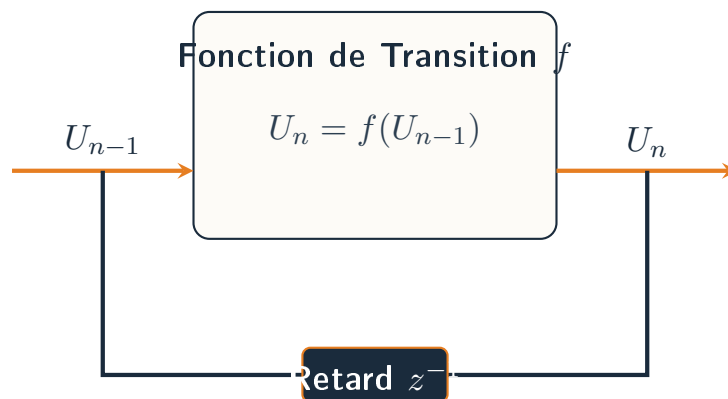


mathbacai.com

LIVRET D'EXCELLENCE

Sciences de l'Informatique (4^{ème} Secondaire)

CHAPITRE 4 : LES ALGORITHMES RÉCURRENTS



Un condensé tout-en-un de cours d'élite, 40 flashcards interactives et des évaluations types pour le Baccalauréat Tunisien.

Contents

1	Fiche 1 : Le Cours Complet	2
1.1	Introduction et Concept Fondamental	2
1.1.1	Classification des Récurrences	2
1.2	Applications de Récurrence d'Ordre 1 : Méthode de Héron	2
1.2.1	Formulation Mathématique	3
1.2.2	Implémentation Algorithmique et Pascal	3
1.2.3	Tableau d'Évolution (Calcul de $\sqrt{2}$ avec $U_0 = 2$ et $\epsilon = 0.001$)	3
1.3	Applications de Récurrence d'Ordre 2 : La Suite de Fibonacci	4
1.3.1	Formulation Mathématique	4
1.3.2	Implémentation par Glissement de Variables	4
1.3.3	Tableau de Trace (Calcul de F_5)	4
1.4	Calcul de Séries et de Sommes Récurrentes	5
1.4.1	Approximation de la constante e (Exponentielle de 1)	5
2	Fiche 2 : Le Recueil de Questions-Réponses	6
3	Fiche 3 : Quiz d'Évaluation & Fiches Mémo	13
3.1	Quiz d'Évaluation (10 Questions Types Bac)	13
3.2	Le Grand Mémento de l'Excellence	16
3.3	Les 5 Règles d'Or de la Récurrence	16

1. Fiche 1 : Le Cours Complet

1.1 Introduction et Concept Fondamental

Un algorithme est dit **récurrent** s'il permet de calculer les termes successifs d'une suite ou les éléments d'un ensemble de données en exploitant une relation de dépendance entre le terme actuel (U_n) et un ou plusieurs termes précédents ($U_{n-1}, U_{n-2}, \dots$).

Contrairement aux algorithmes récursifs qui utilisent la pile d'exécution du système pour sauvegarder des états en attente, l'algorithme récurrent s'exécute de manière purement **itérative**. Il utilise un jeu de registres simples ou de variables temporaires en mémoire vive pour opérer le glissement des valeurs (calcul en place ou dynamique), ce qui le rend optimal en termes de complexité spatiale $\mathcal{O}(1)$.

1.1.1 Classification des Récurrences

- a) **Récurrence d'ordre 1** : Chaque nouveau terme dépend uniquement du terme immédiatement précédent.

$$U_n = f(U_{n-1}) \quad \text{avec } U_0 \text{ donné.}$$

- b) **Récurrence d'ordre 2** : Chaque nouveau terme dépend des deux termes précédents.

$$U_n = f(U_{n-1}, U_{n-2}) \quad \text{avec } U_0, U_1 \text{ donnés.}$$

- c) **Récurrence d'ordre k** : Le calcul fait intervenir les k derniers termes connus.

Définition de la Récurrence

Un algorithme récurrent traduit un modèle mathématique discret de façon itérative. Il est composé de :

- **L'Initialisation** : Assignation des valeurs de départ aux variables (les premiers termes de la suite).
- **Le Noyau de Glissement** : Un ensemble de variables tampons qui permettent de mettre à jour les états précédents pour calculer le terme suivant lors de chaque itération.
- **La Condition d'Arrêt** : Elle peut être définie par un nombre déterminé d'itérations (N) ou par un critère de convergence (précision ϵ).

1.2 Applications de Récurrence d'Ordre 1 : Méthode de Héron

La méthode de Héron (ou méthode de Newton) permet d'approcher la racine carrée d'un nombre réel positif A avec une convergence quadratique extrêmement rapide.

1.2.1 Formulation Mathématique

La suite de Héron est définie par :

$$U_n = \frac{1}{2} \left(U_{n-1} + \frac{A}{U_{n-1}} \right) \quad \text{avec } U_0 > 0$$

L'arrêt s'effectue lorsque la différence absolue entre deux termes successifs est inférieure à une précision fixée ϵ :

$$|U_n - U_{n-1}| < \epsilon$$

1.2.2 Implémentation Algorithmique et Pascal

Algorithme : Methode_Heron

```
Fonction Heron (A : Reel ; eps : Reel) : Reel
Déclaration : u_prec, u_act : Reel
Début
  u_prec <- A
  Initialisation de U0 : u_act <- 0.5 * (u_prec + A / u_prec)
  TantQue Abs(u_act - u_prec) >= eps Faire
    u_prec <- u_act
    u_act <- 0.5 * (u_prec + A / u_prec)
  FinTantQue
Heron <- u_act
Fin
```

Traduction Pascal

```
Function Heron(A, eps : Real) : Real;
Var u_prec, u_act : Real;
Begin
  u_prec := A;
  u_act := 0.5 * (u_prec + (A/u_prec));
  While Abs(u_act - u_prec) >= eps Do
    u_prec := u_act;
    u_act := 0.5 * (u_prec + (A/u_prec));
  End;
Heron := u_act;
End;
```

1.2.3 Tableau d'Évolution (Calcul de $\sqrt{2}$ avec $U_0 = 2$ et $\epsilon = 0.001$)

Itération	$U_{préc}$	U_{act}	$ U_{act} - U_{préc} $	Condition $ \Delta \geq \epsilon$
Initialisation	-	2.00000	-	-
1	2.00000	1.50000	0.50000	Vrai ($0.5 \geq 0.001$)
2	1.50000	1.41667	0.08333	Vrai ($0.083 \geq 0.001$)
3	1.41667	1.41422	0.00245	Vrai ($0.00245 \geq 0.001$)
4	1.41422	1.41421	0.00001	Faux ($0.00001 < 0.001$)

1.3 Applications de Récurrence d'Ordre 2 : La Suite de Fibonacci

La suite de Fibonacci est le cas le plus célèbre de récurrence d'ordre 2, où chaque terme est la somme des deux précédents.

1.3.1 Formulation Mathématique

$$F_n = F_{n-1} + F_{n-2} \quad \text{avec} \quad F_0 = 0, F_1 = 1$$

1.3.2 Implémentation par Glissement de Variables

Pour calculer F_n de manière itérative, nous utilisons trois variables : 'a' (F_{k-2}), 'b' (F_{k-1}) et 'c' (F_k). À chaque étape, après calcul de 'c', nous décalons les valeurs : le contenu de 'b' passe dans 'a', et celui de 'c' passe dans 'b'.

Algorithme : Fibonacci_iteratif

```
Fonction Fibonacci (N : Entier) : Entier
Déclaration : a, b, c, i : Entier
Début Si N = 0 Alors
  Fibonacci <- 0
SinonSi N = 1 Alors Fibonacci <- 1
Sinon a <- 0 F0 b <- 1 F1
Pour i de 2 à N Faire
  c <- a + b
  a <- b
  Glissement b <- c
  Glissement
FinPour Fibonacci <- c
FinSi
Fin
```

Traduction Pascal

```
Function Fibonacci(N : Integer) : Integer;
Var a, b, c, i : Integer;
Begin If N = 0 Then
  Fibonacci := 0
Else If N = 1 Then Fibonacci := 1
Else Begin a := 0; b := 1;
For i := 2 To N
Do Begin c := a + b; a := b; b := c; End;
Fibonacci := c; End; End;
```

1.3.3 Tableau de Trace (Calcul de F_5)

Itération	Indice i	Variable 'a' (F_{i-2})	Variable 'b' (F_{i-1})	Variable 'c' (F_i)
Init	-	0	1	-
1	2	1	1	1
2	3	1	2	2
3	4	2	3	3
4	5	3	5	5

Le résultat retourné pour $N = 5$ est donc bien **5**.

1.4 Calcul de Séries et de Sommes Récursives

Certaines fonctions mathématiques comme e^x , $\cos(x)$, ou $\ln(x)$ s'approchent via des sommes infinies (séries de Taylor). En informatique, on calcule la somme partielle des N premiers termes. Pour optimiser le calcul et éviter d'évaluer de grands factoriels et puissances de manière indépendante, on utilise une relation de récurrence entre le terme actuel (T_k) et le précédent (T_{k-1}).

1.4.1 Approximation de la constante e (Exponentielle de 1)

La fonction exponentielle évaluée en 1 est définie par :

$$e = \sum_{k=0}^{+\infty} \frac{1}{k!} = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \dots$$

On pose $T_k = \frac{1}{k!}$. On remarque qu'il existe une relation de récurrence simple :

$$T_k = \frac{1}{k!} = \frac{1}{(k-1)! \times k} = T_{k-1} \times \frac{1}{k} \quad \text{avec} \quad T_0 = 1$$

Ainsi, le calcul de la somme cumulée $S_N = \sum_{k=0}^N T_k$ s'effectue en temps linéaire $\mathcal{O}(N)$ sans aucun dépassement de capacité d'entier pour le factoriel !

Algorithme : Approximation_E

```
Fonction Approximer_E (N : Entier) : Reel
Déclaration : S, T : Reel k : Entier
Début S <- 1.0 S0 = T0 T <- 1.0 T0 = 1/0!
Pour k de 1 à N Faire T <- T / k Terme suivant récurent
S <- S + T Somme FinPour
Approximer_E <- S Fin
```

Traduction Pascal

```
Function Approximer_E(N : Integer) : Real;
Var S, T : Real; k : Integer;
Begin S := 1.0; T := 1.0;
For k := 1 To N Do Begin T := T/k; S := S + T; End;
Approximer_E := S; End;
```

2. Fiche 2 : Le Recueil de Questions-Réponses

Question 1 : Qu'est-ce qu'une relation de récurrence en algorithmique ?

Réponse d'Excellence :

Une relation de récurrence est une formule mathématique qui exprime le terme général d'une suite en fonction d'un ou de plusieurs termes précédents (ex: $U_n = f(U_{n-1})$). En algorithmique, elle est implémentée de manière itérative en réutilisant les valeurs calculées aux étapes précédentes pour obtenir les termes futurs.

Question 2 : Quelle est la différence fondamentale entre récursivité et récurrence ?

Réponse d'Excellence :

La récursivité est un concept de programmation où une fonction s'appelle elle-même (stockage des contextes d'appel dans la pile système). La récurrence est un modèle mathématique résolu de manière purement itérative (utilisation d'une boucle simple et de variables temporaires en mémoire vive), ce qui la rend beaucoup plus économe en mémoire.

Question 3 : Pourquoi la récurrence itérative est-elle préférable à la récursivité en termes de performance spatiale ?

Réponse d'Excellence :

L'algorithme récurrent itératif s'exécute dans un espace mémoire de taille fixe $\mathcal{O}(1)$, car il n'utilise pas de pile d'appels. Un algorithme récursif non terminal nécessite un espace mémoire de taille $\mathcal{O}(n)$ pour stocker les registres d'activation de la pile d'exécution, risquant ainsi de provoquer un débordement de pile (Stack Overflow).

Question 4 : Comment implémente-t-on une récurrence d'ordre 2 en informatique ?

Réponse d'Excellence :

Elle s'implémente par le mécanisme de glissement de variables. On utilise trois variables en mémoire vive : une pour représenter U_{n-2} , une pour U_{n-1} et une pour le terme calculé U_n . Après chaque calcul du nouveau terme, on effectue un décalage séquentiel ($U_{n-2} \leftarrow U_{n-1}$ puis $U_{n-1} \leftarrow U_n$).

Question 5 : Qu'est-ce que le "noyau de glissement" dans un algorithme récurrent ?

Réponse d'Excellence :

C'est la séquence d'instructions, placée généralement en fin de boucle itérative, qui assure la mise à jour des variables de mémoire pour l'étape suivante. Par exemple, dans la suite de Fibonacci :

'a := b; b := c;' constitue le noyau de glissement.

Question 6 : Pourquoi est-il nécessaire de faire attention à l'ordre des affectations lors du glissement des variables ?

Réponse d'Excellence :

Si l'on écrase la variable représentant le terme le plus ancien avant de l'avoir transférée, la logique est brisée. Dans Fibonacci, faire 'b := c; a := b;' détruit la valeur de 'b' (qui est remplacée par 'c') avant qu'elle ne soit copiée dans 'a'. L'ordre correct est impératif : 'a := b; b := c;'.

Question 7 : Comment définit-on la condition d'arrêt dans un algorithme récurrent de type approximation ?

Réponse d'Excellence :

Elle est définie par une condition de convergence relative à une précision absolue minimale ϵ (epsilon). On compare l'écart absolu entre deux termes successifs : $|U_{act} - U_{prec}| < \epsilon$. Dès que cet écart est franchi, la boucle s'arrête.

Question 8 : Qu'est-ce qu'une boucle infinie dans le cadre d'une approximation par récurrence, et comment l'éviter ?

Réponse d'Excellence :

Une boucle infinie survient si la suite de récurrence ne converge pas, ou si la précision ϵ demandée dépasse la capacité de stockage physique des réels (limite de précision machine). Pour l'éviter, il faut toujours implémenter un compteur de sécurité limitant le nombre maximal d'itérations autorisées.

Question 9 : Quelle est la complexité temporelle et spatiale de l'algorithme de Fibonacci itératif ?

Réponse d'Excellence :

La complexité temporelle est linéaire $\mathcal{O}(n)$ car on calcule chaque terme de 2 à N exactement une fois à l'aide d'une boucle simple. La complexité spatiale est constante $\mathcal{O}(1)$ car seules trois variables de taille fixe ('a', 'b', 'c') sont instanciées en mémoire vive.

Question 10 : Quelle est la complexité temporelle de l'algorithme de Fibonacci récursif naïf ?

Réponse d'Excellence :

Elle est exponentielle de l'ordre de $\mathcal{O}(2^n)$. Cela s'explique par la répétition massive de calculs redondants (un arbre d'appels binaires redondants), rendant l'approche récursive naïve inutilisable pour de grandes valeurs de N .

Question 11 : Comment la méthode de Héron calcule-t-elle la racine carrée d'un nombre A ?

Réponse d'Excellence :

Elle utilise la relation de récurrence d'ordre 1 : $U_n = \frac{1}{2}(U_{n-1} + \frac{A}{U_{n-1}})$. C'est l'application de la méthode de Newton à la fonction $f(x) = x^2 - A$, dont la convergence vers $\sqrt{A}$ est quadratique.

(le nombre de décimales exactes double à chaque itération).

Question 12 : Quelle valeur de départ U_0 choisit-on classiquement pour la méthode de Héron ?

Réponse d'Excellence :

On choisit généralement $U_0 = A$ (la valeur dont on cherche la racine) ou $U_0 = \frac{A}{2}$. Toute valeur strictement positive garantit théoriquement la convergence vers $\sqrt{A}$.

Question 13 : Pourquoi l'utilisation de factoriels directs est-elle déconseillée dans le calcul des séries de Taylor au Baccalauréat ?

Réponse d'Excellence :

Parce que la fonction factorielle grandit de manière extrêmement rapide (13! dépasse la capacité des entiers 32 bits standard). Pour éviter ce dépassement de capacité (Overflow), on calcule le terme général de manière récurrente par multiplication successive : $T_k = T_{k-1}/k$.

Question 14 : Quelle est la relation de récurrence permettant de calculer le terme de la série de e^x de façon optimale ?

Réponse d'Excellence :

Le terme d'ordre k est $T_k = \frac{x^k}{k!}$. La relation de récurrence s'écrit : $T_k = T_{k-1} \times \frac{x}{k}$ avec $T_0 = 1$. Cette formulation n'utilise que des divisions et multiplications de nombres réels et élimine le calcul explicite de factorielles et de puissances.

Question 15 : En Pascal, quel type de données doit-on privilégier pour calculer des limites de suites ?

Réponse d'Excellence :

Il faut impérativement utiliser le type 'Real' (ou 'Double' / 'Extended' pour une précision accrue) afin d'éviter les erreurs d'arrondi numérique machine lors des divisions.

Question 16 : Comment traduit-on l'expression mathématique de la valeur absolue en Pascal ?

Réponse d'Excellence :

On utilise la fonction prédéfinie 'Abs(X)' qui retourne la valeur absolue de l'expression numérique passée en paramètre, qu'elle soit de type entier ou réel.

Question 17 : Pourquoi est-il crucial d'initialiser correctement toutes les variables dans un algorithme récurrent ?

Réponse d'Excellence :

Puisque le terme suivant se construit à partir des valeurs des variables actuelles, toute absence d'initialisation (qui laisse des valeurs aléatoires résiduelles en mémoire vive) fausse l'intégralité de la suite numérique calculée.

Question 18 : Quel est l'intérêt d'étudier la convergence d'une suite récurrente en informatique ?

Réponse d'Excellence :

L'étude de la convergence permet d'assurer que l'algorithme de calcul numérique s'arrêtera bien au bout d'un nombre fini d'étapes (preuve de terminaison) et fournira une approximation valide.

Question 19 : Qu'est-ce qu'une suite récurrente croisée (ou imbriquée) ?

Réponse d'Excellence :

C'est un ensemble de deux ou plusieurs suites qui dépendent mutuellement les unes des autres. Exemple : $A_n = \frac{A_{n-1} + B_{n-1}}{2}$ et $B_n = \sqrt{A_{n-1} B_{n-1}}$ (Moyenne arithmético-géométrique).

Question 20 : Comment implémente-t-on le calcul des suites croisées de façon itérative ?

Réponse d'Excellence :

On utilise des variables tampons temporaires lors de la phase de transition pour éviter d'écraser la valeur d'une suite avant que l'autre ne l'ait utilisée pour son propre calcul (ex: 'temp := A; A := (A+B)/2; B := Sqrt(temp*B);').

Question 21 : Quelle boucle de contrôle Pascal utilise-t-on lorsque le nombre d'itérations d'une suite récurrente est connu à l'avance ?

Réponse d'Excellence :

On utilise la boucle 'For .. To .. Do' (compteur d'itérations fixe), idéale pour calculer le N -ème terme d'une suite sans condition de précision.

Question 22 : Quelle boucle de contrôle Pascal privilégie-t-on pour les calculs d'approximation à précision ϵ ?

Réponse d'Excellence :

On privilégie la boucle 'While .. Do' (ou 'Repeat .. Until'), car le nombre total d'itérations nécessaires pour converger vers la précision voulue n'est pas connu a priori.

Question 23 : Quelle est la différence syntaxique majeure entre 'While' et 'Repeat' en Pascal dans un processus récurrent ?

Réponse d'Excellence :

La boucle 'While' teste sa condition d'entrée *avant* d'exécuter le bloc d'instructions (le bloc peut donc s'exécuter 0 fois). La boucle 'Repeat' exécute le bloc au moins une fois et s'arrête dès que la condition finale devient vraie.

Question 24 : Comment peut-on optimiser la recherche d'un élément dans une suite récurrente monotone ?

Réponse d'Excellence :

Si la suite est strictement croissante et que l'on cherche une valeur cible, on peut arrêter la recherche dès que le terme actuel dépasse cette cible, évitant ainsi des calculs inutiles.

Question 25 : Comment peut-on simuler un algorithme récurrent à l'aide d'un tableau ?

Réponse d'Excellence :

En déclarant un tableau de taille N . On remplit l'élément ' $T[i]$ ' en exploitant l'élément ' $T[i-1]$ '. Bien que simple, cette méthode consomme inutilement $\mathcal{O}(N)$ d'espace mémoire, contrairement au glissement de variables qui utilise $\mathcal{O}(1)$.

Question 26 : Qu'est-ce que l'erreur d'arrondi machine (ou précision de la machine) ?

Réponse d'Excellence :

C'est la limite physique de représentation des nombres réels en binaire. Sur un type 'Real' standard, la précision est d'environ 10^{-11} à 10^{-15} . Demander un $\epsilon < 10^{-16}$ provoquera une boucle infinie.

Question 27 : Que se passe-t-il si la valeur de départ U_0 de la méthode de Héron est négative ?

Réponse d'Excellence :

La suite convergera vers la racine négative $-\sqrt{A}$. Si $U_0 = 0$, la division par zéro provoquera un plantage immédiat de l'exécution.

Question 28 : Comment peut-on calculer de manière récurrente la somme des carrés des N premiers entiers ?

Réponse d'Excellence :

On pose $S_k = S_{k-1} + k^2$ avec $S_0 = 0$. On résout cela à l'aide d'une boucle simple de 1 à N .

Question 29 : Quelle est l'expression de récurrence pour calculer X^N de manière itérative ?

Réponse d'Excellence :

On initialise $P = 1.0$, et on répète la relation de récurrence $P_k = P_{k-1} \times X$ exactement N fois.

Question 30 : Quelle est la complexité d'une recherche dichotomique dans un tableau généré par récurrence ?

Réponse d'Excellence :

Si le tableau est généré de manière ordonnée, la recherche dichotomique s'exécute en temps logarithmique $\mathcal{O}(\log N)$.

Question 31 : Comment gère-t-on les exposants négatifs dans un calcul de puissance récurrente ?

Réponse d'Excellence :

On calcule la puissance de la valeur absolue de l'exposant, puis on retourne l'inverse du résultat ($1/P$).

Question 32 : Peut-on convertir tout algorithme récurrent en un algorithme récursif ?

Réponse d'Excellence :

Oui, tout algorithme récurrent (itératif) admet une écriture récursive équivalente. La récursion sera de nature terminale (Tail Recursive), ce qui est optimal.

Question 33 : La réciproque est-elle vraie : peut-on dérécursiver tout algorithme récursif sous forme récurrente simple ?

Réponse d'Excellence :

Oui, mais la conversion nécessite parfois l'utilisation explicite d'une structure de pile de données gérée manuellement par le programmeur si la récursion n'est pas terminale.

Question 34 : Quelle relation de récurrence régit la suite de Syracuse ?

Réponse d'Excellence :

Pour un entier donné U_{n-1} : si U_{n-1} est pair alors $U_n = U_{n-1}/2$, sinon $U_n = 3 \times U_{n-1} + 1$.

Question 35 : Quelle est la conjecture de Syracuse associée à cette suite ?

Réponse d'Excellence :

Elle conjecture que pour tout entier de départ strictement positif, la suite récurrente finit toujours par atteindre le cycle d'oscillation stationnaire 4, 2, 1.

Question 36 : Comment implémente-t-on le calcul du nombre de combinaisons C_n^p de façon récurrente (Triangle de Pascal) ?

Réponse d'Excellence :

On utilise la relation de récurrence d'ordre 2 bidimensionnelle : $C_n^p = C_{n-1}^p + C_{n-1}^{p-1}$ avec les conditions aux limites $C_n^0 = C_n^n = 1$.

Question 37 : Qu'est-ce qu'une suite récurrente linéaire d'ordre k ?

Réponse d'Excellence :

C'est une suite dont le terme U_n s'exprime comme une combinaison linéaire des k termes précédents : $U_n = a_1 U_{n-1} + a_2 U_{n-2} + \dots + a_k U_{n-k}$.

Question 38 : Comment assure-t-on la robustesse d'une fonction récurrente face à des entrées invalides ?

Réponse d'Excellence :

En intégrant des préconditions en début de fonction (ex: vérifier que le radicande A dans Héron est strictement supérieur à zéro) et en levant des exceptions ou retournant des codes d'erreur.

Question 39 : Pourquoi est-il formellement interdit de modifier la variable de contrôle d'une boucle 'For' à l'intérieur de la boucle en Pascal ?

Réponse d'Excellence :

Parce que cela détruit la prévisibilité de l'arrêt de la boucle, corrompt la gestion interne du registre

d'index par le compilateur et conduit à des comportements indéterminés.

Question 40 : Comment la récurrence est-elle valorisée dans l'épreuve pratique du Baccalauréat tunisien ?

Réponse d'Excellence :

Elle est évaluée sur la capacité de l'élève à traduire un formalisme mathématique complexe en une boucle itérative optimisée en place (sans tableaux inutiles) et stable numériquement.

3. Fiche 3 : Quiz d'Évaluation & Fiches Mémo

3.1 Quiz d'Évaluation (10 Questions Types Bac)

Question 1 (QCM)

La suite de Fibonacci calculée de manière itérative possède une complexité spatiale de :

- A) $\mathcal{O}(2^n)$
- B) $\mathcal{O}(n)$
- C) $\mathcal{O}(1)$

Correction détaillée :

La bonne réponse est **C**. L'algorithme itératif n'utilise qu'un nombre constant de variables simples ('a', 'b', 'c') en mémoire vive, peu importe la valeur de N . L'espace alloué en mémoire est donc stable et indépendant de N .

Question 2 (Vrai ou Faux)

La méthode de Héron permet d'approcher la valeur de la racine carrée d'un nombre réel négatif.

Correction détaillée :

Faux. Le calcul de la racine carrée n'est défini que pour des nombres réels positifs ou nuls. De plus, une valeur négative pour la variable A provoquerait des oscillations infinies ou des valeurs non réelles non gérables par le type 'Real' standard.

Question 3 (QCM)

Pour éviter une boucle infinie lors d'une approximation par récurrence à précision ϵ , il faut :

- A) Augmenter systématiquement la valeur de ϵ
- B) Ajouter une condition limitant le nombre d'itérations maximales
- C) Remplacer la boucle 'While' par une boucle 'For'

Correction détaillée :

La bonne réponse est **B**. L'ajout d'une borne supérieure sur le nombre de boucles (un compteur d'itérations de sécurité) garantit impérativement l'arrêt du programme même si la suite diverge ou si la précision ϵ est inférieure à la précision machine.

Question 4 (Vrai ou Faux)

En Pascal, l'instruction 'a := b; b := c;' produit le même résultat que 'b := c; a := b;'.

Correction détaillée :

Faux. L'ordre des affectations est crucial. Dans la seconde séquence, 'b' est écrasé par 'c', puis 'a' reçoit cette nouvelle valeur 'b' (donc 'c'). La valeur initiale de 'b' est définitivement perdue, détruisant le mécanisme de glissement.

Question 5 (QCM)

Soit le terme d'une série $T_k = \frac{x^k}{k!}$. La relation de récurrence optimale liant T_k à T_{k-1} est :

A) $T_k = T_{k-1} \times \frac{x}{k}$

B) $T_k = T_{k-1} \times x$

C) $T_k = T_{k-1}/k$

Correction détaillée :

La bonne réponse est **A**. Par développement mathématique : $\frac{x^k}{k!} = \frac{x^{k-1}}{(k-1)!} \times \frac{x}{k} = T_{k-1} \times \frac{x}{k}$. Cette écriture évite de calculer séparément x^k et $k!$, éliminant les risques de dépassement d'entiers (Overflow).

Question 6 (Vrai ou Faux)

La boucle 'Repeat .. Until' s'exécute au moins une fois, même si sa condition d'arrêt est déjà satisfaite au départ.

Correction détaillée :

Vrai. Contrairement à 'While', la boucle 'Repeat' est à condition de sortie *post-testée*. Les instructions du bloc interne sont toujours exécutées une première fois avant que la condition placée après le 'Until' ne soit évaluée.

Question 7 (QCM)

Quelle est la complexité temporelle du calcul de Fibonacci récursif naïf ?

A) $\mathcal{O}(n)$

B) $\mathcal{O}(2^n)$

C) $\mathcal{O}(\log n)$

Correction détaillée :

La bonne réponse est **B**. Chaque appel engendre deux autres appels récursifs (F_{n-1} et F_{n-2}), formant un arbre binaire complet d'appels redondants dont la taille croît de façon exponentielle.

Question 8 (Vrai ou Faux)

La suite de Syracuse pour $U_0 = 7$ finit par atteindre la valeur 1.

Correction détaillée :

Vrai. Conformément à la conjecture de Syracuse, toutes les valeurs entières testées à ce jour convergent vers le cycle d'attraction final $4 \rightarrow 2 \rightarrow 1$.

Question 9 (QCM)

Quel type d'accès est intrinsèquement utilisé lors du calcul séquentiel d'une récurrence ?

- A) Accès direct
- B) Accès séquentiel
- C) Accès aléatoire

Correction détaillée :

La bonne réponse est **B**. Pour évaluer le terme U_n , on doit calculer obligatoirement tous les termes intermédiaires un par un du premier au dernier, ce qui correspond à un traitement séquentiel strict.

Question 10 (Vrai ou Faux)

L'utilisation de variables globales est fortement recommandée pour effectuer le glissement de valeurs dans une fonction récurrente en Pascal.

Correction détaillée :

Faux. L'usage de variables globales crée des effets de bord nuisibles et nuit à la modularité et la réutilisabilité du code. Toutes les variables de calcul d'une fonction récurrente doivent être déclarées localement ('Var' interne).

3.2 Le Grand Mémento de l'Excellence

Ce tableau récapitule les principales formules de récurrence du programme de Sciences de l'Informatique, leurs conditions et initialisations.

Suite / Algorithme	Relation Mathématique	Initialisations & Arrêt
Suite de Fibonacci	$F_n = F_{n-1} + F_{n-2}$	$F_0 = 0, F_1 = 1$. Arrêt à l'indice N .
Méthode de Héron ($\sqrt{A}$)	$U_n = \frac{1}{2} \left(U_{n-1} + \frac{A}{U_{n-1}} \right)$	$U_0 = A$. Arrêt si $ U_n - U_{n-1} < \epsilon$.
Série Exponentielle (e^x)	$T_k = T_{k-1} \times \frac{x}{k}$	$T_0 = 1, S_0 = 1$. Arrêt à la précision ϵ .
Suite de Syracuse	$U_n = U_{n-1}/2$ (pair) ou $3U_{n-1} + 1$ (impair)	$U_0 = \text{Entier choisi}$. Arrêt si $U_n = 1$.
Série du Sinus ($\sin(x)$)	$T_k = -T_{k-1} \times \frac{x^2}{2k(2k+1)}$	$T_0 = x, S_0 = x$. Arrêt à l'ordre N .

3.3 Les 5 Règles d'Or de la Récurrence

1. **Initialiser sans exception** : Toujours affecter une valeur de départ correcte à l'ensemble des variables participant à la récurrence sous peine de propager des valeurs mémoires résiduelles.
2. **Vérifier l'ordre du glissement** : Respecter rigoureusement la chronologie des décalages temporels (le plus ancien terme est écrasé en premier).
3. **Sécuriser les approximations** : Toujours ajouter un garde-fou (un compteur maximal de boucles) dans les structures de boucle 'While' basées sur une précision réelle ϵ afin de prévenir les boucles infinies.
4. **Éviter les factorielles directes** : Préférer l'évaluation récurrente des termes généraux pour s'affranchir du risque de dépassement de capacité sur les grands entiers (Overflow).
5. **Privilégier les variables locales** : Déclarer toutes les variables de calcul et de glissement au sein de la fonction locale pour garantir la propreté structurelle de votre code Pascal.